TECHNICAL WHITEPAPER · 2026

‹ › Mesh.

Structural source code compression for LLM context efficiency — Mesh Compression Engine.

74%	3.9×	$3.90
avg token reduction	context gain per 128k window	vs $15.00/MTok input · Claude Opus 4.6

THE PROBLEM

As context utilization increases, LLM retrieval accuracy degrades — measurably.

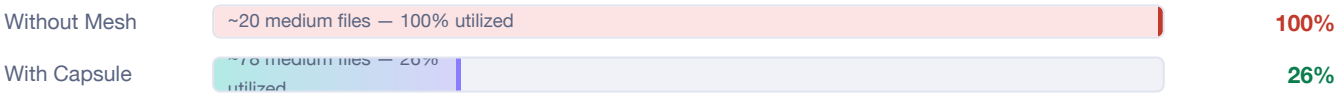
Large language models operate with a fixed context window. This is not a soft limit — it creates a hard accuracy ceiling as utilization approaches 100%.

NIAH — Needle In A Haystack

Standard retrieval benchmark (Liu et al., 2023, "*Lost in the Middle*"): a specific fact is placed at varying positions in a long document; the model must retrieve it.

CONTEXT UTILIZATION	RETRIEVAL ACCURACY
< 60%	~95–98%
60–80%	~75–85%
80–95%	~55–70%
> 95%	~40–60%

The degradation is positional: facts in the middle of the context are retrieved less reliably than those at the beginning or end. At 100k+ token contexts the effect is amplified.



SWE-bench

Industry benchmark for AI coding agents solving real GitHub issues. Top models (Claude Opus 4.6, GPT-4o, Gemini 3.1 Pro) resolve approximately **40–55%** of tasks as of early 2026. Root cause analysis of failures identifies one leading pattern: **the agent lacked sufficient codebase context.**

Current tools use embedding-based retrieval to select files, then send them raw. This selects what gets loaded — it does not reduce what each file costs. As codebase size grows, coverage per context window shrinks.

HOW CAPSULE WORKS

Structural compression with selective recovery — not summarization.

Capsule processes source files **before** they enter any LLM context window. The original file is always fully reconstructable on demand.

Pipeline stages

STAGE	OPERATION	OUTPUT
1 — Parse	Language-aware AST extraction	Structural components: declarations, signatures, type annotations, doc comments, control flow markers
2 — Compress	Implementation body reduction	Bodies replaced with structural stubs; non-structural tokens minimized
3 — Encode	Descriptor assembly	Compact plain-text structural descriptor preserving semantic skeleton
4 — Selective Recovery	On-demand body restoration	Full implementation body injected inline when model identifies it as needed

What is preserved

- All export and declaration names
- All function and method signatures (parameters, return types)
- Type definitions, interfaces, and type aliases
- Class structure and inheritance hierarchy
- Doc comment content
- Control flow markers (try/catch presence, loop presence, async/await markers)
- Module imports and dependency references

Activation threshold

Files below ~200 tokens receive pass-through (format overhead exceeds savings). In production codebases, ~95% of files by token count are in the compression range. The threshold has no material effect on total context reduction.

PERFORMANCE

Production pipeline · 5 file types · 6 size tiers · 50 files per tier

TypeScript, YAML, SQL, HTML, Markdown — real numbers from the production Capsule pipeline.

FILE SIZE	TOKEN REDUCTION	SAVING
~200B (xs) — pass-through		+10%
~1 KB (small)		–83%
~5 KB (medium)		–95%
~18 KB (large)		–98.5%
~50 KB (XL)		–99.4%
~100 KB (XXL)		–99.9%
Average (small–XXL)		–74% → 3.9×

Context window coverage at 128k tokens

Without Mesh	~20 medium files — 100% utilized	100%
With Capsule	~78 medium files — 26% utilized	26%

Why compression ratio improves with file size

Implementation bodies scale with file size; structural skeletons do not. A 100 KB TypeScript file has approximately the same structural skeleton size as a 5 KB file — the extra 95 KB is almost entirely implementation bodies. The compression ratio therefore improves asymptotically as files grow larger.

ECONOMICS

74% cost reduction per query — same files, same model, same result quality.

Per-query cost (Claude Opus 4.6 — $15.00/MTok input)

INPUT TYPE	TOKENS SENT	COST PER QUERY
Raw source (100 medium files)	1,000,000	$15.00
Capsule compressed (same 100 files)	**~260,000**	**$3.90**

Scale projection (10,000 users · 50 queries/day · 50k avg context tokens)

METRIC	RAW	CAPSULE
Total tokens/day	25,000,000,000	~6,500,000,000
API cost/day (Claude Opus 4.6)	$375,000	**~$97,500**
Monthly API cost	$11,250,000	**~$2,925,000**
Monthly savings	**$8,325,000 (–74%)**	

The cost curve is flat with Capsule — adding more files to context costs near-zero marginal tokens once those files are in the compression range.

SYSTEM DESIGN

Retrieval selects files. Compression reduces what each file costs.

```
User query
    |
    ▼
Workspace indexer        ← scans project directory, builds file registry
    |
    ▼
File selector            ← embedding-based semantic retrieval, returns ranked candidates
    |
    ▼
Capsule compression      ← structural compression per file (parallel workers)
    |
    ▼
Context assembler        ← packs compressed descriptors, targets ~65% utilization
    |
    ▼
Model API call           ← Claude / Gemini / GPT (provider-agnostic)
    |
    ▼
Selective recovery       ← restores full implementation bodies on model request
    |
    ▼
Response + diff output
```

Retrieval vs. Compression — not mutually exclusive

PROPERTY	EMBEDDING RETRIEVAL	CAPSULE COMPRESSION
What reaches the model	Raw file content (full token cost)	**Compressed structural descriptor**
Token cost scales with	File count × file size	**Near-constant (structural skeleton size)**
Context coverage (128k)	~20 medium files	**~78 medium files**
Codebase size sensitivity	High	**Low — ratio improves with size**
Additive with retrieval	N/A	**Yes — operates post-retrieval**

Gateway / Worker architecture

Compression runs in stateless worker processes — horizontally scalable, independent of the UI and API gateway. Worker throughput scales linearly with worker count.

Model compatibility

Capsule output is plain text in a structured format. No fine-tuning, no special tokenizer. Compatible with any transformer-based LLM: Claude, Gemini, GPT-4o, o1, o3, o4-mini. Provider switching is runtime-configurable — no re-indexing required.

File type support (v1)

LANGUAGE	PRESERVED STRUCTURAL ELEMENTS
TypeScript / JavaScript	Exports, class declarations, function signatures, type definitions, interfaces, generics
Python	Module-level declarations, class/function signatures, type hints, docstrings
YAML / JSON	Top-level keys, schema structure (values compressed to type markers)
SQL	Table/view/function definitions, column names and types, index definitions
HTML / JSX	Component tree structure, prop signatures, slot structure
Markdown	Heading hierarchy, code block presence, link structure

Go, Rust, Java, C#, Ruby, PHP planned for Capsule v2.